App Project 2: LED Intensity Controller (Group Project)

Due Date: As per D2L Dropbox

Assignment: Using the Microcontroller and your experience from developing driver functions developed so far, you will design an "*LED Light Control" App* to control the brightness/ intensity of LEDs connected to your microcontroller. You will also log the brightness setting on your PC with a Python program that you must design and implement. The requirements that your project must satisfy are described in this brief. The push buttons are defined as in the previous assignments and should be connected to the same pins as before. We define a push button "click" as a press and release of a button.

User input(s)	Output(s)
If PB1 is clicked	After programming the microcontroller, if PB1 is clicked, it should put the System in **ON MODE.** In this mode, an LED1 should turn on. Turning the potentiometer should adjust the LED intensity to between full brightness and 0% (LED off). The variable brightness must be achieved by using Pulse width modulation of the signal supplied to the LED1 as described below. If PB1 is held down for a "long press" in **ON MODE**, LED1 should turn off, and LED2 should turn on, with its brightness now controlled by the potentiometer. Doing another long press of PB1 should swap LEDs again. If PB1 is clicked again, the System should enter **OFF MODE,** i.e., turn off the LED and put the system in low-power Idle() mode.
If PB2 is clicked	While in **ON MODE**, if PB2 is pressed once, it should blink the currently active LED. When the LED is on during the blink, it should be at the intensity level corresponding to the current ADC reading. When the LED is off during the blink, the intensity is 0. The blinking should happen at approx. 500 ms intervals (0.5 s on, 0.5 s off). If you turn the potentiometer, the intensity while blinking should adjust accordingly. While in **OFF MODE**, if PB2 is pressed once, it should blink the LED at 100% intensity level at approx. 500 ms intervals (0.5 s on, 0.5 s off). The LED blinking should stop if PB2 is clicked again in either mode.
If PB3 is clicked	If PB3 is clicked during the **ON MODE** (non-blink) or **ON MODE** (blink), the microcontroller should start to transmit the intensity level and ADC reading to a PC over UART. On the PC, you should design and implement a Python script to receive the UART transmissions. For a period of **1 minute**: 1. Capture and Store the intensity levels (0-100%) of the LED and the ADC readings from the microcontroller, in a single CSV file with appropriate indexing and column names. You should time-stamp the readings appropriately. 2. After 1 minute, you should produce a graph/plot of the intensity levels (0-100%) vs. time (in seconds) and another graph of the ADC reading vs. time (i.e., two separate graphs, that could be subplots).

	Your graphs **must** have an appropriate title and axes labels. The generated CSV file should be named as per your group name and have proper index numbers and column names. In order to capture the LED blinking properly, you will need to transmit data at a rate of more than 2 Hz. Note that while the LED is off (during a blink), we would expect the "intensity" readings to be 0%. Clicking PB3 again, or leaving the ON MODE, should stop the UART transmissions. While transmitting, it must be possible for a user to go back and forth between blink and non-blink modes and have the transmitted data/generate graph show the transition. Note that UART transmissions should only occur after PB3 has been clicked as described above.

Additional info:

Implement the above controller in C as a finite-state machine-based controller, to run on your hardware kit. **You must not use the Output Compare/PWM peripheral in this project. Polling of PORTx or TMRx directly, or corresponding flags that you create, instead of interrupt-driven programming, may lose points.** Note that you may use multiple timer peripherals or the 32-bit timer configuration at your discretion. You might recall that **there are three timers available to you**. You will need to navigate the datasheet by yourselves, accordingly. Your implementation should be as power-efficient as possible. You may implement "polling-like" behavior to read your ADC and determine when to send data over UART.

Function names: You may use any convention when naming functions or organizing code. A state diagram of your design is required as part of your submission. Use microcontroller-specific registers, bit names, and flags (to jump between states) wherever applicable in the state diagram. Your C code **must** accurately reflect your FSM and vice versa.

Display instructions: All displays on the PC should be using Python. Note that display functions carried out at 32 kHz (300 Baud) can affect timer delays. Your code should account for such delays when producing delays specified in the table above. You may choose to use whatever clock speed(s) available to you as you see fit.

Interrupts: Interrupt ISR names are provided in the lecture slides. As specified in lectures, IO (CN interrupts) are triggered on rising and falling edges and due to any bounce effects of the push buttons that cause several high-to-low and low-to-high fluctuations at the Microcontroller input. Your code should filter out any such effects. Note: the "click" behavior descriptions above suggest that your system must not change state just because a button is pressed (so a press and hold down should not cause a behavior change), but the button being released should.

Your submission must satisfy the following in addition to achieving all requirements of this project and delivering all deliverables to gain full marks: (i) You do not call functions in an ISR (this includes no UART messages in the ISR), unless you can prove that the function(s) called are reentrant (ii) You do not have infinite loops in an ISR, (iii) You do not add delays to the ISR, (iv) You do not poll the input ports (i.e., your program should not be repeatedly testing (reading) the value of the PORTA or PORTB, (v) Your submission should not have "glitches" (e.g., pushing the buttons in an arbitrary order at arbitrary times should not result in unexpected behavior), (vi) your C program does not poll status flags related to the timers or inputs, and (vii) there should not be any visual flickering of the LED. You may submit a project that does not satisfy all requirements but will only receive partial credit. All members of your group must be in attendance at the demo.

B. Tan

You are responsible for regularly checking D2L for announcements (e.g., via the News tool) regarding the project. As with any engineering project, requirements and guidance are subject to change at the teaching team's discretion. If you require clarification, it is your responsibility to discuss your thoughts with the instructor during the timetabled class times.

Pulse Width Modulated Signals (PWM):

PWM signals are commonly used in many mechatronic applications like light and motor control, etc., and look like Figures 1 and 2 below. You will program the microcontroller to generate a PWM signal from Pin 12, connected to an LED via a 1K-ohm current limiting resistor. The duty cycle should be controllable based on the Potentiometer input.

(Figure Sources: realdigital.org)

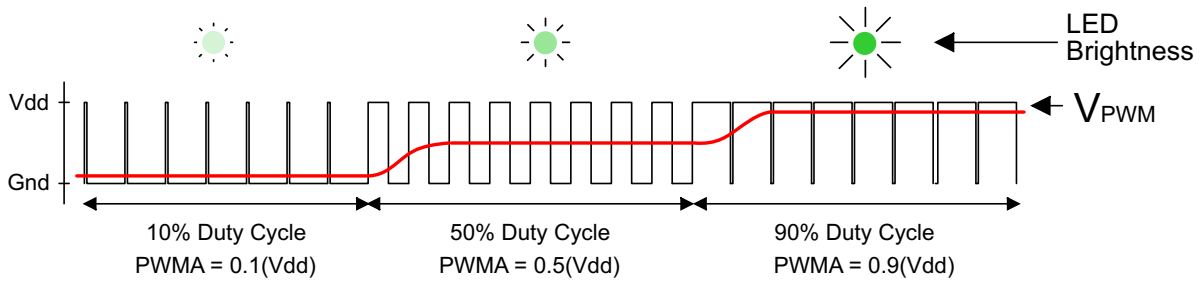


Figure 1: Relationship between PWM Duty cycle and LED brightness or intensity levels

Vpwm = Duty Cycle/100 * VDD, where
Vpwm: PWM voltage or average voltage of a PWM Signal
VDD: Voltage supplied on a Microcontroller pin

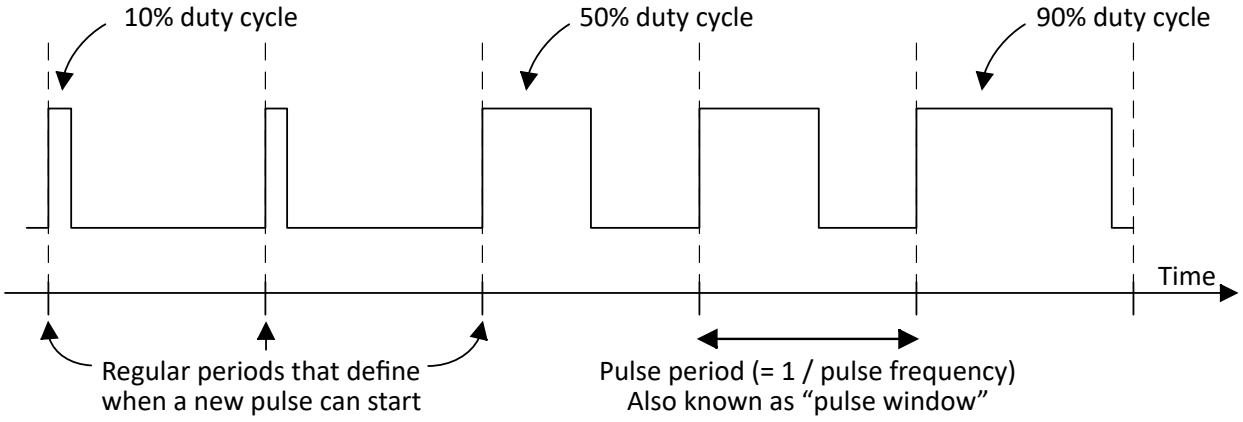


Figure 2: PWM Signal

Pulse Frequency: You can use any pulse frequency for your app. However, the pulse frequency should be high enough so that the resulting LED flicker is not visible to the human eye but, at the same time, not too high to consume excessive power. Use trial and error when coding your app to determine an appropriate Pulse Frequency.

Deliverables:

This is a group project. Deliverables include:

1. **Zipped up file of the MPLAB project uploaded to D2L**. MPLAB projects can be zipped up by right-clicking on the project and selecting "Package." The zipped project is saved in the same project folder created by the user. Make sure your driver code is commented properly. **This must be submitted before the demos.**

2. **A single pdf document uploaded to D2L** showing the following:
    a. Names and UCIDs of all students in the group at the top of the document
    b. **A State diagram** showing the working of your code. Use microcontroller-specific register and bit names, and code-specific function names/labels wherever application in the state diagram.
    c. A report with a description of all important elements of your implementation for both C and Python (supported by screenshots/snippets, or other diagrams and flow charts, as appropriate). Remember to include details about how you implemented elements, such as the principle behind your implementation of the PWM functionality, ADC reading, and UART transmissions, as well as data "cleaning" in Python.

3. **Python Code of your UART receiving program uploaded to D2L** in .py format as a single text file. **You must provide instructions in a README file so that a member of the teaching team can run your Python code.**

4. **Single CSV of an example data capture uploaded to D2L** showing time, intensity levels and ADC reading corresponding to a test run you performed before submission. **This file should be properly indexed and show column names for time, intensity level, and ADC reading.**

5. **A completed group work form uploaded to D2L. This must be endorsed (signed) by all members of the group. Note: Individual marks might be adjusted to account for imbalances in the group participation. All students must have made technical contributions to the project.**

6. **Perform a live demo of your project to members of the teaching team in the lab during the lab times of the week starting Dec. 1.**
    a. To perform your demo, you will first clear your MPLAB X IDE projects list, download your submission from D2L, import it back into MPLAB X IDE, and program your microcontroller in front of the assessor.
    b. You will have up to **12 minutes** to demonstrate your project, which includes the time to download your D2L submission, etc.
    c. Demo of the C and Python code and hardware operation showing all states, preferably with reference to your state machine. Each group member should be prepared to answer questions related to the working of their app project.
    d. Your group will be scheduled for a time during your timetabled lab session or lecture times. A schedule will be published closer to the time.

7. **At the conclusion of your demo, ALL GROUP MEMBERS must return the hardware kit.**

Note: you are not allowed to share laptops/code/hardware between groups. Plagiarism of code or report is not allowed. Please monitor your email inbox and D2L in case there are any changes to the deliverables, clarifications to the project, or other communication from your instructor. As with any engineering project, requirements and guidance are subject to change at the teaching team's discretion.

B. Tan

Project Rubric

	Fails to meet expectations	Minimally meets expectations	Adequately Meets Expectations	Exceeds Expectations
Peripheral Configuration and Use Program Logic	None of the peripherals and states working correctly Does not provide evidence of appropriate program flow **0 points**	Some of the peripherals and states working correctly to implement the functionality in the brief Provides evidence of attempting to use C control statements but has some errors or does not cover all scenarios in the brief **Up to 12 points**	Most of the peripherals and states working correctly to implement the functionality in the brief Provides evidence of attempting to use C control statements to cover most but not all scenarios in the brief or has some errors **Up to 16 points**	All peripherals and states correctly working to implement the functionality in the brief Provides evidence of appropriate C control statements and implements all scenarios correctly as in the brief and other scenarios in a justified manner **Up to 20 points**
Response to task	Does not address all aspects of the brief. **0 points**	Submission addresses most, but not all aspects of the brief. Several elements of the submission are unclear, vague, or inaccurate. **Up to 3 points**	Submission addresses most, but not all aspects of the brief. Few elements of the submission are unclear, vague, or inaccurate. **Up to 4 points**	Convincingly addresses all aspects of the brief in an accurate, clear, and concise manner. **Up to 5 points**
Efficiency and usability	Does not provide evidence of using power-efficient features (e.g., Idle) or usability **0 points**	Provides some limited evidence of using power-efficient features (e.g., Idle, clock switching) and usability **Up to 3 points**	Provides some evidence of using power-efficient features (e.g., Idle, clock switching) and usability (robustness of changing timer value, resetting the timer, etc.) **Up to 4 points**	Provides strong evidence of using power-efficient features **appropriately** (e.g., Idle, clock switching) and usability (e.g., robustness of changing timer value, resetting the timer, etc.) **Up to 5 points**
Code Comments/ Structure	No evidence of code commenting or reasonable variable names. **0 points**	Some evidence of code comments, but infrequent or incomplete. **Up to 3 points**	Evidence that most elements of the code are commented. **Up to 4 points**	Evidence that all important elements of the code are commented meaningfully. **Up to 5 points**
E State Machine Diagram and Description	No state machine diagram uploaded or unclear diagram. **0-1 points**	Several states missing or incorrect. **Up to 2 points**	Some states missing or incorrectly drawn. **Up to 4 points**	Proper state machine showing all states and using Microcontroller and code specific register/flag/function names. Implementation clearly matches design. **Up to 5 points**
Report/ Documentation	No report provided or report with vague details as to the logic of the design and implementation. **0 points**	Basic report provided that outlines at least some of the important elements of the design and implementation. May have some errors of expression (e.g., typographical errors, inappropriate tone) **Up to 6 points**	Basic report that provides details of some of the important elements of the design and implementation, supported with relevant screenshots/diagrams/snippets of code, as appropriate. May have some errors of expression (e.g., typographical errors, inappropriate tone) **Up to 8 points**	Detailed report that provides details of all the important elements of the design and implementation supported with relevant screenshots/diagrams/snippets of code, as appropriate. Professionally presented. **Up to 10 points**
Teamwork	Not all team members have a reported contribution to the submission **0 points**	All team members have reported that they have contributed to the submission, but areas of contribution are unclear **Up to 3 points**	All team members have reported that they have contributed to the submission, but areas of contribution are a little vague **Up to 4 points**	All team members have reported that they have contributed to the submission, with areas of contribution clearly specified and with appropriate levels of balance across team members. **Up to 5 points**
				Total (55):

Tentative indicators:

MME – basic LED blink, solid ON and UART transmissions (intensity and ADC), correctly working button presses/states, correctly working Python code
AME – basic LED blink, dimmable ON and UART transmissions (intensity and ADC), correctly working button presses/states, correctly working Python code
EE – basic LED blink, dimmable LED blink, dimmable ON and UART transmissions (intensity and ADC), correctly working button presses/states, correctly working Python code